UNITED STATES INTERNATIONAL UNIVERSITY – AFRICA

School of Science and Technology

Department of Computing

Student Project Log-Book

Smart Health Symptom Checker and Doctor Recommendation System

Submitted by

Amy Wanjugu Njau – 669008

Course: SWE3090XA – Software Engineering Final Project

Supervisor: Mr Fredrick Ogore

Date of submission: August 2026

Summer Semester 2026

Table of Contents

Student Information

Field	Detail
Name of student	Njau, Amy Wanjugu
Registration number	669008
Faculty	School of Science and Technology
Course of study	SWE3090XA - Software Engineering Final Project
Stage / year of study	Final year
Project undertaken	Smart Health Symptom Checker and Doctor Recommendation System
Name of project supervisor	Mr Fredrick Ogore
Duration of the project	12 weeks, Summer Semester 2026

Proposed System

A cross-platform mobile application that accepts user-reported symptoms, returns a ranked list of probable conditions together with the symptoms that produced each ranking, recommends an appropriate medical specialist, and locates nearby qualified providers using the user's real-time position.

Proposed system features:

- Structured symptom entry from a curated catalogue.
- A rule-based diagnostic engine using a weighted symptom-condition matrix.
- Explainable results: every ranking shows the symptoms that produced it.
- Specialist recommendation derived from the top-ranked condition.
- Location-aware nearby provider recommendation.
- A guidance disclaimer attached to every diagnosis response.
- Graceful behaviour when location permission is denied.

Week One: Project Conceptualization

Day	Description of activities	New skills learnt
Mon	Reviewed the project brief and shortlisted three candidate problem areas in digital health. Settled on the gap between symptom onset and reaching the right clinician.	Framing a problem statement around a measurable gap rather than a technology.
Tue	Desk research on healthcare access in low- and middle-income settings; gathered WHO material on health-worker shortages and primary care.	Locating and judging the reliability of secondary sources.
Wed	Reviewed existing symptom checkers: WebMD, Ada Health, Babylon, Buoy and K Health. Recorded what each does and does not do.	Structured competitor review using fixed comparison criteria.
Thur	Drafted the problem statement and the main objective; wrote the five specific objectives.	Writing objectives that are specific enough to be tested later.
Fri	Met the supervisor to confirm scope. Agreed the system is a guidance tool, not a diagnostic device, and that the boundary must be explicit throughout.	Scoping a safety-sensitive project and stating exclusions early.

Week Two: Project Proposal

Day	Description of activities	New skills learnt
Mon	Built the comparative analysis table scoring existing systems on symptom analysis, specialist routing, location awareness, explainability and low-connectivity suitability.	Turning a literature review into a decision-useful comparison.
Tue	Identified the four consistent gaps and wrote the unique contributions the project would make.	Distinguishing a gap in the literature from a gap in the market.
Wed	Read the BMJ audit study by Semigran and colleagues on symptom-checker accuracy; noted the implications for safety framing.	Reading a clinical audit study and extracting design constraints from it.
Thur	Chose a rule-based approach over machine learning and wrote the justification: explainability, absence of a labelled clinical dataset, and correctability.	Justifying a technical choice against project constraints rather than fashion.
Fri	Compiled and submitted the project proposal. Agreed the Agile approach with the supervisor.	Structuring a proposal so each section answers a marking criterion.

Week Three: Project UML and System Design

Day	Description of activities	New skills learnt
Mon	Drew the three-tier architecture: Flutter client, Node and Express API hosting the engine, and a data and location tier.	Reasoning about tiers in terms of what changes independently.
Tue	Produced the context diagram, treating the system as one process with the user, the Maps Platform and the data store as external entities.	Level-0 data-flow modelling.
Wed	Decomposed the system into five processes for the level-1 data flow diagram and identified the four data stores.	Level-1 decomposition without leaking implementation detail.
Thur	Modelled the seven entities for the ER diagram. Introduced Condition_Symptom as an associative entity carrying the rule weight.	Using an associative entity to make a many-to-many relationship carry data.
Fri	Reviewed the models with the supervisor. Confirmed that placing the weight on the relationship is what allows the rule base to be tuned without schema change.	Defending a data-model decision on the grounds of future change.

Week Four: Project Design

Day	Description of activities	New skills learnt
Mon	Defined the API contract as the integration boundary: endpoints, request bodies and response shapes, written before any code.	Contract-first design between client and server.
Tue	Designed the scoring rule: sum the weights of reported symptoms, divide by the condition's maximum possible weight to normalise to 0-1.	Normalising scores so conditions with different symptom counts stay comparable.
Wed	Decided the client would hold no diagnostic logic, so the rule base can be corrected without an app release.	Placing logic according to how often it changes and who must trust it.
Thur	Designed the data-store interface so a local file-backed store and a Firestore store are interchangeable.	Programming to an interface to defer an infrastructure decision.
Fri	Sketched the mobile flow: symptom input, ranked results with explanation chips, nearby providers, history and profile.	Designing a flow around one task rather than around screens.

Week Five: Progress Reporting - Backend Foundation

Day	Description of activities	New skills learnt
Mon	Initialised the Node and Express project. Separated `createApp()` from `server.js` so tests can start the app without binding a fixed port.	Structuring an application for testability from the first commit.
Tue	Centralised every environment read and the safety disclaimer in one configuration module.	Keeping configuration auditable in a single file.
Wed	Authored the first version of the knowledge base as declarative JSON: conditions with weighted symptoms.	Expressing domain rules as data rather than as code.
Thur	Built the symptom, specialist and provider catalogues and the local data store.	Separating repository concerns from service logic.
Fri	Hit a Google Drive limitation: `node_modules` cannot live in the synced folder. Wrote a setup script installing dependencies to local disk and pointing Node at them.	Diagnosing an environment constraint and automating around it.

Week Six: Progress Reporting - Diagnostic Engine

Day	Description of activities	New skills learnt
Mon	Implemented `scoreCondition`, returning both the normalised score and the list of symptoms that contributed to it.	Returning the reasoning alongside the result to make output explainable.
Tue	Implemented ranking: filter below the configurable threshold, sort descending, cap the result count.	Making tuning parameters configuration rather than constants in logic.
Wed	Kept the engine a pure function with no input or output, so it can be tested without a server or database.	Purity as a deliberate device for testability.
Thur	Wrote the first unit tests covering full match, partial match, unknown symptoms and ranking order.	Writing tests that express a requirement rather than repeat the implementation.
Fri	Added defensive de-duplication of the symptom set and used `hasOwnProperty` to avoid matching inherited object properties.	Guarding against prototype pollution in user-supplied keys.

Week Seven: Progress Reporting - API and Mobile Start

Day	Description of activities	New skills learnt
Mon	Built the diagnose endpoint and the orchestration service tying engine, specialist lookup, query log and disclaimer together.	Keeping route handlers thin and putting orchestration in a service.
Tue	Added request validation middleware returning 400 with clear messages, and a central error handler that never leaks internals on a server error.	Handling failure centrally instead of at each call site.
Wed	Scaffolded the Flutter application: theme, models and the tabbed shell.	Structuring a Flutter app around typed models mapped from JSON.
Thur	Built the symptom input screen against the live catalogue endpoint.	Driving a UI from server-supplied data rather than hard-coded lists.
Fri	Built the results screen and surfaced the matched symptoms as explanation chips.	Turning an internal data structure into a user-facing explanation.

Week Eight: Progress Reporting - Location and Providers

Day	Description of activities	New skills learnt
Mon	Implemented the provider service with two interchangeable sources selected by configuration.	Designing for a swappable external dependency.
Tue	Implemented haversine distance and ranked the bundled providers by proximity.	Great-circle distance calculation and when it is accurate enough.
Wed	Added the Google Nearby Search path, called server-side so the Maps key never reaches the client.	Protecting a billable API key by placing the call server-side.
Thur	Added a time-to-live cache for provider lookups to contain both latency and billing.	Caching keyed on the parameters that actually determine the result.
Fri	Implemented location permission handling on the client, with a default location and a visible banner when permission is denied.	Treating a denied permission as a supported path rather than an error.

Week Nine: Progress Reporting - Integration

Day	Description of activities	New skills learnt
Mon	Connected the full journey end to end: symptoms, conditions, specialist, nearby providers.	Integrating subsystems through a contract instead of point to point.
Tue	Added the history and profile tabs backed by an in-app store.	Managing shared state across tabs without losing it on switch.
Wed	Added live directions from a provider card, opening the device map application.	Handing off to a platform application through a URL scheme.
Thur	Wrote the integration tests that drive the HTTP API with the engine and stores wired together.	Testing across a boundary without mocking the thing under test.
Fri	Ran the complete flow on the emulator and fixed the defects found.	End-to-end testing against a running system rather than against intent.

Week Ten: Progress Reporting - Testing and Documentation

Day	Description of activities	New skills learnt
Mon	Completed the automated suite covering the engine, the endpoints and both validation failures.	Reading a test suite as evidence for a requirement.
Tue	Ran the acceptance scenarios including denied location and simulated network failure.	Designing test cases around failure as well as success.
Wed	Produced the architecture, sequence and entity-relationship figures for the reports.	Producing figures that match the system as built.
Thur	Wrote the System Integration Report covering approach, data flow, protocols and security.	Writing a technical report against a fixed structure.
Fri	Reviewed progress with the supervisor and agreed the remaining priorities.	Reporting status against a plan honestly, including what slipped.

Week Eleven: Progress Reporting - Reports and Authentication

Day	Description of activities	New skills learnt
Mon	Built a repeatable document pipeline turning markdown sources into branded PDF and Word reports.	Automating a deliverable so formatting cannot drift between documents.
Tue	Rebuilt the System Integration Report through the pipeline and corrected figures that no longer matched the delivered system.	Checking that a document's figures still describe the system they claim to.
Wed	Added Firebase Authentication: token verification on the server, sign-in and registration on the client.	Verifying a signed token against a provider's rotating public keys.
Thur	Established the rule that a supplied token is always verified even when authentication is optional, and wrote a test for it.	Recognising that an optional check must not become a skipped check.
Fri	Scoped query history to the signed-in account and recorded the account id on each query-log entry.	Scoping stored data to an identity without storing more than is needed.

Week Twelve: Progress Reporting - Final Project Finalization

Day	Description of activities	New skills learnt
Mon	Ran a ranking evaluation over twelve symptom vignettes and recorded the result with its limitation stated.	Reporting a measurement together with what it does not prove.
Tue	Wrote the end-of-semester report covering design, implementation, testing, results and conclusions.	Sustaining one argument across a long technical document.
Wed	Audited every claim in the report against the code and corrected those that overstated what was built.	Verifying documentation against the running system rather than against memory.
Thur	Prepared the defence presentation and rehearsed the demonstration path.	Presenting a technical system to a mixed audience.
Fri	Finalised the repository, documentation and submission package.	Preparing work so somebody else can run it without assistance.

Summary of the Period

The twelve weeks moved the project from a problem statement to a working, tested system. The first four weeks established the problem, reviewed existing solutions, and produced the architecture and data models. Weeks five to nine delivered the backend, the diagnostic engine, the mobile client and the location tier, integrated through a single API contract. Weeks ten to twelve covered testing, documentation, authentication and the final reports.

The technical work that mattered most was the decision to keep the diagnostic engine a pure function scoring a declarative knowledge base. It made the highest-risk component the easiest to test, it kept every result explainable down to named symptoms, and it allowed the rule base to grow without any change to engine code.

The principal difficulty was not technical. It was learning to separate what the system does from what it can be claimed to do. The knowledge base is curated rather than clinically validated, and the discipline of saying so in the interface, in the API response, and in every report is the part of the project most likely to carry into professional work.

Supervisor Weekly Report

For completion by the project supervisor.

General comments on the student's progress:

Name of the supervisor: Mr Fredrick Ogore

Department / unit: Department of Computing, School of Science and Technology

Date: …… Signature: ……